Java / J2EE Backend Senior Professional

Complete Interview Preparation Series

19 Chapters · ~930 Pages · Deep Q&A; + Code Examples + Exercises

Phase 1: Foundations	**Phase 2: Framework & Data**	**Phase 3: Security**
Phase 4: Capabilities	**Phase 5: Perf & Architecture**	**Phase 6: Infrastructure**

Targeting: Google · Amazon · Meta · Microsoft · Top-Tier Tech Companies

For Professionals with 15 – 20+ Years of Experience

19 Chapters	**~930 Pages**	**6 Phases**	**4 Exercise Types**

Table of Contents

Total: 19 chapters · 945 pages

About This Series

This 19-chapter series is a complete interview preparation guide for senior Java / J2EE backend professionals targeting principal and staff engineer roles at top-tier technology companies such as Google, Amazon, Meta, and Microsoft. The series is structured in the exact order you encounter these topics when building real production software — from core language foundations through application frameworks, security, capabilities, performance, architecture, and finally infrastructure and cloud deployment.

Each chapter combines three elements: (1) concise documentation explaining how the technology works internally, (2) deep interview Q&A; with full answers and Java code examples, and (3) four types of exercises — coding challenges, debug/fix problems, design problems, and MCQ / scenario quizzes. Answers are targeted at 15-20+ years of experience and cover both senior and principal/staff level depth.

The series is delivered as 19 individual PDFs — one per chapter — so you can focus on the areas most relevant to your interview preparation without being overwhelmed by a single document.

Exercise Types Included in Every Chapter			
Coding Challenges	Debug / Fix Problems	Design Problems	MCQ / Scenario Quiz

Phase 1: Foundations

4 chapters · ~210 pages

01

Core Java & JVM Internals

Java fundamentals: primitives vs wrappers, autoboxing, pass-by-value, immutability, String pool, StringBuilder vs StringBuffer, equals()/hashCode() contract. Collections Framework (in-depth): Full interface/class hierarchy — Iterable > Collection > List/Set/Queue/Deque; Map hierarchy (separate). LIST: ArrayList (dynamic array, 1.5x growth, O(1) get), LinkedList (doubly-linked + Deque), CopyOnWriteArrayList (thread-safe snapshot); imports, all List/ArrayList methods (add, get, set, remove, indexOf, subList, sort, toArray, List.of), complete code examples. SET: HashSet (hash table, O(1)), LinkedHashSet (insertion order), TreeSet (red-black tree, sorted, NavigableSet — floor/ceiling/headSet/tailSet), EnumSet (bit-vector); imports, all Set/SortedSet/NavigableSet methods, set operations (union/intersection/difference), code examples. MAP: HashMap (bucket array, treeification at bin-size 8, load factor 0.75), LinkedHashMap (insertion/access order, LRU cache pattern), TreeMap (red-black tree, NavigableMap — floorKey/ceilingKey/pollFirstEntry), EnumMap, WeakHashMap, IdentityHashMap, ConcurrentHashMap (CAS + per-bin lock, no full-table lock); imports, all Map methods (put/get/remove, getOrDefault, putIfAbsent, computeIfAbsent, merge, forEach, entrySet), complete code examples. QUEUE/DEQUE: PriorityQueue (min-heap, custom Comparator), ArrayDeque (preferred Stack/Queue), BlockingQueue variants (LinkedBlockingQueue, ArrayBlockingQueue); all offer/poll/peek methods. Fail-fast (modCount) vs fail-safe iterators. Collection utility class (sort, binarySearch, unmodifiableList, synchronizedList, frequency, disjoint). Choosing the right collection — decision table. Generics: type erasure, bridge methods, wildcards (PECS rule), generic methods. Exception handling: hierarchy, checked vs unchecked, try-with-resources, multi-catch, custom exceptions, anti-patterns. Java 8-21 features: lambdas, Stream API (lazy evaluation, Collectors), Optional, method references, functional interfaces, Date/Time API, records, sealed classes, pattern matching, virtual threads (Project Loom), structured concurrency. Multithreading: thread lifecycle, synchronized, wait/notify, volatile, JMM happens-before, ReentrantLock/ReadWriteLock/StampedLock, AtomicInteger/CAS/ABA, ThreadLocal, ExecutorService/ThreadPoolExecutor, CompletableFuture, ForkJoinPool, virtual threads. JVM internals: class loading delegation model, linking phases, JIT (C1/C2 tiers), memory areas (heap, Metaspace, stack), GC algorithms (G1, ZGC, Shenandoah), GC tuning flags, thread dump/heap dump analysis with JFR and async-profiler.

Est. 75 pages

02

Design Patterns & Design Principles

Design Principles (OOP & SOLID): Single Responsibility (one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes must be substitutable — covariant return types, no precondition strengthening), Interface Segregation (many specific interfaces > one fat interface), Dependency Inversion (depend on abstractions not concretions). Additional principles: DRY (Don't Repeat Yourself), KISS (Keep It Simple), YAGNI (You Aren't Gonna Need It), SoC (Separation of Concerns), LoD / Law of Demeter (talk only to immediate friends), Composition over Inheritance, Program to Interfaces. Cohesion vs Coupling — measuring and improving each. Creational Patterns: Singleton (double-checked locking with volatile, enum singleton, holder idiom — thread safety analysis for each), Factory Method (decouple object creation, subclass decides), Abstract Factory (families of related objects without specifying concrete classes), Builder (fluent API, immutable objects, separating construction from representation — Java examples with inner static Builder class), Prototype (clone() contract, deep vs shallow copy). Structural Patterns: Adapter (class adapter via inheritance vs object adapter via composition, Java example bridging legacy and new API), Bridge (decouple abstraction from implementation, avoid class explosion), Composite (tree structures, treat leaves and composites uniformly — file system example), Decorator (add behaviour dynamically without subclassing — InputStream/OutputStream chain as canonical example), Facade (simplified interface to complex subsystem), Flyweight (share intrinsic state, externalise extrinsic — Integer.valueOf cache, String pool), Proxy (JDK dynamic proxy with InvocationHandler, CGLIB subclass proxy, virtual proxy, protection proxy, remote proxy). Behavioral Patterns: Strategy (define family of algorithms, make them interchangeable — Comparator as Strategy), Observer (publish-subscribe, Java EventListener, Spring ApplicationEvent), Command (encapsulate request as object, undo/redo support), Template Method (define skeleton, defer steps to subclasses — HttpServlet as example), Chain of Responsibility (pass request along handler chain — Servlet Filter chain, Spring Security filter chain), State (object behaviour changes with state — order state machine example), Iterator (sequential access without exposing underlying structure — java.util.Iterator contract), Visitor (add operations to object structure without modifying it), Mediator (centralise complex communication — chat room example), Memento (capture and restore object state — undo mechanism). Enterprise & Architectural Patterns: Repository (abstract data access behind collection-like interface, Spring Data as implementation), Unit of Work (track changes, flush as single transaction — JPA EntityManager), Service Layer (application logic boundary between presentation and domain), DTO (Data Transfer Object — mapping entities to API responses, MapStruct), DAO (Data Access Object — raw JDBC vs JPA), Value Object (immutable, equality by value — Java records as ideal Value Objects). Architectural Patterns: Layered (Presentation / Service / Repository / Domain), Hexagonal / Ports and Adapters (domain core with pluggable adapters for DB/HTTP/MQ), MVC (Spring MVC mapping), Event-Driven (loosely coupled event producers and consumers). Anti-Patterns: God Class, Spaghetti Code, Golden Hammer, Premature Optimisation, Anemic Domain Model, Service Locator (vs DI). Pattern selection guide: problem-to-pattern decision table with Java examples throughout.

Est. 55 pages

03

Data Structures & Algorithms in Java

Covers Big-O complexity analysis with a Java collection operation reference table, arrays and strings (sliding window, two-pointer, prefix sum), linked lists, stacks and queues, trees and BSTs (traversals, AVL), heaps and priority queues (k-way merge, top-K), hash tables (Java HashMap as implementation), graphs (BFS, DFS, Dijkstra, topological sort, union-find), sorting algorithms (merge, quick, heap sort with Java implementations), dynamic programming (memoisation vs tabulation, classic patterns), recursion and backtracking, and segment/Fenwick trees for range queries.

Est. 40 pages

04

Build Tools: Maven & Gradle

Covers Maven's POM, GAV coordinates, lifecycle phases, plugin system, dependency scopes, transitive dependency resolution, nearest-wins conflict rule, dependencyManagement, BOMs, and multi-module Reactor builds. Covers Gradle's task DAG, incremental builds, build cache, Kotlin DSL, configurations (implementation vs api), version catalogs, dependency locking, and multi-project builds. Includes a full Maven vs Gradle comparison (performance, flexibility, learning curve, CI integration), fat JAR packaging, publishing to Artifactory/Nexus, and build performance optimisation for CI pipelines.

Est. 40 pages

Phase 2: Application Framework & Data Layer

3 chapters · ~150 pages

05

Spring Framework: Internals-First

Covers IoC and DI fundamentals (BeanFactory vs ApplicationContext), all three configuration styles (XML, annotation, Java), @Autowired resolution internals via AutowiredAnnotationBeanPostProcessor, bean scopes and the prototype-in-singleton problem, the full Spring bean lifecycle (all 12 steps of ApplicationContext.refresh()), Spring proxy mechanism (JDK dynamic proxy vs CGLIB - when each is used, self-invocation pitfall), Spring AOP (advice types, pointcut expressions, ordering), @Transactional internals (proxy-based, all propagation and isolation levels), Spring MVC request pipeline (DispatcherServlet through ViewResolver), Spring Boot auto-configuration (@Conditional annotations, custom starters), Spring WebFlux reactive model (Mono/Flux, backpressure), and Spring Security internals (filter chain, OAuth2, method-level security).

Est. 55 pages

06

JPA & ORM Internals

Covers JPA spec vs Hibernate vs Spring Data JPA, EntityManagerFactory and EntityManager internals, persistence context as identity map, all four entity lifecycle states and transitions, dirty checking and flush modes, first-level cache (session cache) and batch processing implications, second-level cache (providers, concurrency strategies), all association types with fetch strategies, the N+1 problem and every solution (JOIN FETCH, @EntityGraph, @BatchSize, DTO projections), inheritance mapping strategies (SINGLE_TABLE, JOINED, TABLE_PER_CLASS) with trade-offs, optimistic and pessimistic locking with @Version, bulk operations and batch insert performance, JPQL vs Criteria API vs native queries, advanced Hibernate features (@Formula, @Where, @Filter, @NaturalId, JSON columns), and Hibernate 6 / Jakarta Persistence 3 changes.

Est. 55 pages

07

Databases & SQL

Covers JDBC (Connection, PreparedStatement, connection pooling), SQL deep dive (joins, subqueries, CTEs, window functions), B-tree vs LSM tree index structures, clustered vs non-clustered indexes, covering indexes and composite index prefix matching, query execution plan analysis (EXPLAIN ANALYZE), ACID properties and how databases guarantee them, all four isolation levels and what anomalies each prevents, MVCC (how readers don't block writers), deadlock detection and prevention, database design (normalisation, when to denormalise), NoSQL databases (Cassandra, MongoDB, DynamoDB, Redis), schema migration with Flyway and Liquibase, and read replica and sharding strategies.

Est. 40 pages

Phase 3: Application Security

1 chapters · ~45 pages

08

Security: Auth, Certificates, SSO & API Security

Covers cryptography fundamentals (AES, RSA, hashing, digital signatures, HMAC), HTTPS and TLS internals (TLS 1.3 handshake step by step, cipher suites, forward secrecy), X.509 certificates and PKI (certificate chain, CA trust model, CRL vs OCSP, KeyStore/TrustStore in Java, mTLS), cookies in depth (HttpOnly, Secure, SameSite, CSRF relationship), authentication fundamentals (password storage with bcrypt, MFA/TOTP, Spring Security), authorisation models (RBAC, ABAC, ReBAC), OAuth2 deep dive (all grant types, PKCE), OpenID Connect (ID token, discovery endpoint), JWT (structure, RS256 vs HS256, alg:none vulnerability, revocation), Single Sign-On (SAML 2.0, OIDC-based SSO, session federation, Single Logout, Keycloak, LDAP/Active Directory), OWASP Top 10 with Java-specific mitigations, and API security best practices.

Est. 45 pages

Phase 4: Application Capabilities

4 chapters · ~185 pages

09

Caching: All Levels

Covers the memory hierarchy and latency numbers, L1/L2/L3 CPU cache and false sharing (@Contended in Java), in-process JVM cache with Caffeine (W-TinyLFU eviction algorithm), distributed cache with Redis (data structures, persistence, Cluster, Sentinel, Pub/Sub), Hibernate first-level and second-level cache, database query cache and materialised views, CDN and edge caching (Cache-Control, ETag, surrogate keys, stale-while-revalidate), HTTP and browser cache, reverse proxy cache (Nginx, Varnish, API Gateway caching), multi-level tiered caching architecture (L1 Caffeine + L2 Redis + L3 DB), all five caching strategies (cache-aside, read-through, write-through, write-behind, refresh-ahead), cache invalidation approaches, all eviction policies (LRU, LFU, W-TinyLFU, TTL), and Spring @Cacheable/@CacheEvict integration.

Est. 50 pages

10

Application Search: Elasticsearch & Alternatives

Covers why database LIKE queries fail at scale, inverted index internals (tokenisation pipeline, stemming, synonyms, posting lists), Elasticsearch architecture (cluster, shards, segments, write/read path, NRT indexing, ILM hot/warm/cold tiers), Query DSL in depth (term vs full-text queries, bool query, geo queries, function score), relevance scoring (BM25, field boosting, explain API), Spring Data Elasticsearch integration, keeping Elasticsearch in sync with the database (dual-write, Debezium CDC + Kafka, zero-downtime reindex with aliases), advanced features (autocomplete, fuzzy search, vector/semantic search with kNN), alternative search engines (Solr, OpenSearch, Algolia, Typesense, PostgreSQL tsvector, pgvector), and search architecture patterns (CQRS applied to search, search analytics, personalised ranking).

Est. 45 pages

11

Internationalisation: Multi-Language, Multi-Currency & Multi-Timezone

Covers i18n vs L10n vs Globalisation, Java Locale and Accept-Language header handling, Spring MessageSource and LocaleResolver, resource bundles with pluralisation (ICU MessageFormat), database content localisation strategies, text encoding (UTF-8 everywhere, utf8mb4 in MySQL), bidirectional text (RTL for Arabic/Hebrew, CSS direction, Unicode BiDi algorithm), BigDecimal for money (never double), JSR 354 Moneta library, ISO 4217 currency codes, exchange rate storage and caching, locale-aware currency formatting, rounding modes, the golden rule of timezone handling (store UTC, display local), java.time API in depth (Instant vs ZonedDateTime vs LocalDateTime), DST pitfalls and recurring event scheduling, user timezone detection, Jackson timezone serialisation, NumberFormat and DateTimeFormatter with locale, and Elasticsearch language analysers.

Est. 45 pages

12

Concurrent User Edits: Conflict Detection & Graceful Handling

Covers the lost update problem and silent overwrite (the default broken behaviour), all six conflict handling strategies: Last-Write-Wins (when acceptable), Optimistic Concurrency Control with @Version (end-to-end flow from UI to DB), HTTP ETags and conditional requests (If-Match, 412 Precondition Failed), Pessimistic Locking (record checkout with heartbeat, lock expiry, admin override), Field-Level Merge with three-way diff (JSON Patch, @DynamicUpdate), and Queue-Based Serialisation (Kafka partition key per entity, actor model). Covers graceful error handling - converting OptimisticLockException to structured 409 Conflict with user-facing diff, Spring @ExceptionHandler with RFC 9457 ProblemDetail, UI retry-with-reload patterns, idempotency keys for safe retries, and full end-to-end scenario walkthroughs (hotel double-booking, CRM edit conflict, flash sale).

Est. 45 pages

Phase 5: Performance & Architecture

3 chapters · ~125 pages

13

Software Performance: Sync, Async & Semi-Sync

Covers performance fundamentals (latency vs throughput, percentile metrics, Little's Law, Amdahl's Law), synchronous communication in depth (thread-per-request model, connection pool sizing, cascading latency, C10K problem - pros and cons), asynchronous communication (event-loop model, NIO, Project Reactor, CompletableFuture, async messaging - pros and cons), semi-synchronous patterns (Future with timeout, async-then-poll, request-reply over Kafka, virtual threads as 'sync code with async performance' - pros and cons), threading models compared (thread-per-request vs event loop vs virtual threads vs Fork/Join vs actor model), JVM performance tuning (GC selection, heap sizing, JIT warm-up, GraalVM native, profiling with JFR and flame graphs), database performance (connection pool exhaustion, N+1 detection, read replicas, write batching), and load/stress/soak/spike testing with Gatling and k6.

Est. 45 pages

14

Microservices & Distributed Architecture

Covers microservice decomposition principles (DDD bounded contexts, single responsibility, right-sizing), all inter-service communication patterns (REST, gRPC, async messaging), distributed transactions without 2PC (Saga pattern - choreography vs orchestration, Outbox pattern), API Gateway responsibilities (TLS termination, auth offloading, rate limiting, routing, BFF pattern), service discovery, resilience patterns (Circuit Breaker with Resilience4j, Bulkhead thread pool isolation, Retry with exponential backoff and jitter, Rate Limiter token vs leaky bucket), observability (distributed tracing, structured logging, Micrometer + Prometheus + Grafana, SLI/SLO/SLA), event-driven architecture (event vs command vs query, schema evolution), Kafka deep dive (ISR, exactly-once semantics, consumer groups, KEDA scaling), distributed systems fundamentals (CAP/PACELC, Raft consensus, vector clocks), and CQRS and Event Sourcing.

Est. 45 pages

15

System Design at Scale

Covers scalability patterns (horizontal scaling, stateless services, back pressure), caching architecture in system design context (multi-level, stampede prevention, consistent hashing), load balancing algorithms (Round Robin, Least Connections, IP Hash, Consistent Hashing with virtual nodes), database sharding (range vs hash vs directory-based, resharding challenges, cross-shard queries, hot shard detection), and 10 complete system design walkthroughs with full discussion of trade-offs: URL shortener, distributed rate limiter, Twitter-style feed (push vs pull), Kafka-like message queue, search autocomplete, global key-value store (DynamoDB-style), ride-sharing matching (geospatial indexing), distributed job scheduler (exactly-once semantics), notification system (fan-out, priority queues), and payments processing system (idempotency, reconciliation).

Est. 35 pages

Phase 6: Infrastructure & Deployment

4 chapters · ~230 pages

16

Docker: Images, Containers & Docker Without Kubernetes

Covers containers vs VMs (namespaces, cgroups, union filesystem), Docker architecture (daemon, CLI, containerd, runc, OCI standard), image layers and the overlay2 filesystem (content-addressed SHA256 layers, layer caching strategy), Dockerfile best practices for Java (multi-stage builds reducing image from 800MB to 120MB, exec form vs shell form for signal handling, non-root user, MaxRAMPercentage for container awareness), Docker networking (bridge, host, user-defined bridge with DNS, overlay), volumes (bind mount vs named volume vs tmpfs), Docker Compose for multi-container local development, image security (Trivy scanning, distroless/Chainguard images, image signing with Cosign), Docker in CI/CD (BuildKit, multi-platform builds with buildx), Docker Swarm for simple multi-server orchestration, and a comprehensive answer to 'Can Docker be effective without Kubernetes?' covering Docker Compose, Swarm, AWS ECS, GCP Cloud Run, and Fly.io.

Est. 50 pages

17

Reverse Proxy: Nginx, Ingress & Traffic Management

Covers forward proxy vs reverse proxy (what each faces, why the names), the full set of reverse proxy responsibilities (TLS termination, load balancing, routing, compression, caching, rate limiting, auth offloading, header transformation), Nginx architecture (master/worker, event-driven non-blocking I/O), nginx.conf structure (http/server/location blocks), location matching order, proxy_pass and upstream configuration, SSL setup, rate limiting with limit_req_zone, annotated Nginx config examples for Spring Boot microservices, Kubernetes Service types compared (ClusterIP, NodePort, LoadBalancer, ExternalName, Headless), the Nginx Ingress Controller architecture (how it dynamically generates nginx.conf), a complete annotated Kubernetes architecture showing internal-only services (postgres, redis, workers) vs externally-routed services (booking-api, admin-ui) via Ingress, full Ingress YAML with TLS/path routing/canary/CORS annotations, NetworkPolicy as in-cluster firewall, other Ingress controllers (Traefik, Kong, AWS ALB, GKE Ingress, Gateway API), service mesh (Istio east-west mTLS), and cert-manager for TLS automation.

Est. 50 pages

18

Kubernetes: Everything End-to-End

Covers why Kubernetes exists (desired state reconciliation), full control plane internals (kube-apiserver, etcd/Raft, scheduler, controller-manager, cloud-controller-manager), worker node components (kubelet, kube-proxy, containerd/runc), every workload type (Pod, ReplicaSet, Deployment, StatefulSet, DaemonSet, Job, CronJob, HPA, VPA) with use cases, all Service types, storage (PV, PVC, StorageClass, EBS/PD CSI drivers), ConfigMaps and Secrets (External Secrets Operator for AWS/GCP Secrets Manager), full deployment on AWS EKS (eksctl/Terraform, managed node groups vs Fargate, AWS Load Balancer Controller, IRSA, VPC CNI, Karpenter), full deployment on GCP GKE (Standard vs Autopilot, Workload Identity, GCP HTTP(S) LB Ingress, BackendConfig, NEGs, Cloud Armor), programmatically invoking the Kubernetes API for scaling (Fabric8 Java client - scale Deployments, patch HPA, watch rollout, KEDA ScaledObject for Kafka-lag-based autoscaling with full YAML), resource requests vs limits (QoS classes, Java MaxRAMPercentage), RBAC and NetworkPolicy, observability stack (Prometheus Operator, Fluentd, OpenTelemetry), and Helm.

Est. 65 pages

19

DevOps, CI/CD, AWS & GCP

Covers the 12-Factor App methodology applied to Spring Boot, immutable infrastructure, CI/CD pipeline stages (build, unit test, integration test, security scan, staging, production), trunk-based development vs GitFlow, deployment strategies (blue-green, canary, feature flags, rolling), safe database migrations in CD pipelines, rollback strategies, shift-left testing, AWS services for Java backends in depth (EC2/ECS/EKS/Lambda, S3, RDS/Aurora, DynamoDB, ElastiCache, SQS/SNS/EventBridge/Kinesis, API Gateway, CloudFront, Route 53, VPC/IAM/Cognito/Secrets Manager/KMS, CloudWatch/X-Ray, CodePipeline/CDK), GCP services for Java backends in depth (GKE/Cloud Run/Cloud Functions, Cloud SQL/AlloyDB/Spanner/Firestore/Bigtable/Memorystore, Pub/Sub/Cloud Tasks/Workflows, Cloud Load Balancing/CDN/Cloud Armor, Cloud IAM/Secret Manager, Cloud Build/Cloud Deploy/Artifact Registry, BigQuery/Dataflow), AWS vs GCP equivalent services comparison, SRE fundamentals (error budgets, toil reduction, chaos engineering, blameless post-mortems, RTO vs RPO), and IaC with Terraform for both AWS and GCP.

Est. 65 pages

Series at a Glance

#	Chapter	Phase	Pages
1	Core Java & JVM Internals	*Ph 1: Foundations*	~75
2	Design Patterns & Design Principles	*Ph 1: Foundations*	~55
3	Data Structures & Algorithms in Java	*Ph 1: Foundations*	~40
4	Build Tools: Maven & Gradle	*Ph 1: Foundations*	~40
5	Spring Framework: Internals-First	*Ph 2: Application Framework & Data Layer*	~55
6	JPA & ORM Internals	*Ph 2: Application Framework & Data Layer*	~55
7	Databases & SQL	*Ph 2: Application Framework & Data Layer*	~40
8	Security: Auth, Certificates, SSO & API Security	*Ph 3: Application Security*	~45
9	Caching: All Levels	*Ph 4: Application Capabilities*	~50
10	Application Search: Elasticsearch & Alternatives	*Ph 4: Application Capabilities*	~45
11	Internationalisation: Multi-Language, Multi-Currency & Multi-Timezone	*Ph 4: Application Capabilities*	~45
12	Concurrent User Edits: Conflict Detection & Graceful Handling	*Ph 4: Application Capabilities*	~45
13	Software Performance: Sync, Async & Semi-Sync	*Ph 5: Performance & Architecture*	~45
14	Microservices & Distributed Architecture	*Ph 5: Performance & Architecture*	~45
15	System Design at Scale	*Ph 5: Performance & Architecture*	~35
16	Docker: Images, Containers & Docker Without Kubernetes	*Ph 6: Infrastructure & Deployment*	~50
17	Reverse Proxy: Nginx, Ingress & Traffic Management	*Ph 6: Infrastructure & Deployment*	~50
18	Kubernetes: Everything End-to-End	*Ph 6: Infrastructure & Deployment*	~65
19	DevOps, CI/CD, AWS & GCP	*Ph 6: Infrastructure & Deployment*	~65
	Total		**~945**